

This session we're going to cover:

- Lists
- Dictionary

در پایتون، لیست‌ها به ما امکان می‌دهند چندین مورد را در یک متغیر ذخیره کنیم. برای مثال، اگر بخواهید سن تمام دانش‌آموزان یک کلاس را ذخیره کنید، می‌توانید این کار را با استفاده از یک لیست انجام دهید.

لیست‌ها مشابه آرایه‌ها (آرایه‌های پویایی که امکان ذخیره‌سازی اقلامی با انواع داده‌ای متفاوت را فراهم می‌کنند) در سایر زبان‌های برنامه‌نویسی هستند.

ایجاد یک لیست در پایتون

ما با قرار دادن عناصر درون کروشه [] و جدا کردن آن‌ها با ویرگول، یک لیست ایجاد می‌کنیم. برای مثال

```
In [1]: # a list of three elements
ages = [19, 26, 29]
print(ages)
```

```
[19, 26, 29]
```

فهرست کردن اقلام از انواع مختلف

لیست‌های پایتون بسیار منعطف هستند. همچنین می‌توانیم داده‌هایی با انواع مختلف را در یک لیست ذخیره کنیم. برای مثال

```
In [2]: # a list containing strings, numbers and another list
student = ['Jack', 32, 'Computer Science', [2, 4]]
print(student)

# an empty list
empty_list = []
print(empty_list)
```

```
['Jack', 32, 'Computer Science', [2, 4]]
[]
```

ویژگی لیست‌ها

- **ordered** مرتب‌شده - آن‌ها ترتیب عناصر را حفظ می‌کنند
- **Mutable** تغییرپذیر - اقلام پس از ایجاد، قابل تغییر هستند
- **Allow duplicates** اجازه دادن به مقادیر تکراری - می‌توانند شامل مقادیر تکراری باشند

دسترسی به عناصر لیست

هر عنصر در یک لیست با عددی که (اندیس) (index) نامیده می‌شود، مرتبط است

اندیس اولین آیتم 0، اندیس دومین آیتم 1 و به همین ترتیب است

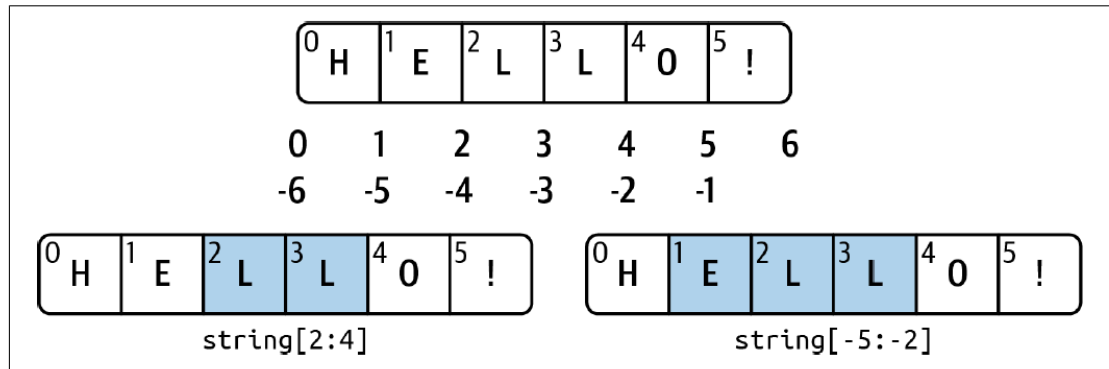


Figure 3-1. Illustration of Python slicing conventions

ما از این اندیس‌ها برای دسترسی به آیتم‌های یک لیست استفاده می‌کنیم. برای مثال

```
In [1]: languages = ['Python', 'Swift', 'C++']

# access the first element
print('languages[0] =', languages[0])

# access the third element
print('languages[2] =', languages[2])
```

```
languages[0] = Python
languages[2] = C++
```

اندیس گذاری منفی

در پایتون، یک لیست می‌تواند اندیس‌های منفی نیز داشته باشد. اندیس آخرین عنصر -1، اندیس عنصر ماقبل آخر -2 و به همین ترتیب است

```
In [2]: languages = ['Python', 'Swift', 'C++']

# access the last item
print('languages[-1] =', languages[-1])

# access the third last item
print('languages[-3] =', languages[-3])
```

```
languages[-1] = C++
languages[-3] = Python
```

برش لیست در پایتون

عملگر برش (slicing) یعنی «:»

اگر نیاز داشته باشیم به بخشی از یک لیست دسترسی پیدا کنیم، می‌توانیم از برش استفاده کنیم. برای مثال

```
In [3]: my_list = ['p', 'r', 'o', 'g', 'r', 'a', 'm']
print("my_list =", my_list)

# get a list with items from index 2 to index 4 (index 5 is not included)
print("my_list[2: 5] =", my_list[2: 5])

# get a list with items from index 2 to index -3 (index -2 is not included)
print("my_list[2: -2] =", my_list[2: -2])

# get a list with items from index 0 to index 2 (index 3 is not included)
print("my_list[0: 3] =", my_list[0: 3])
```

```
my_list = ['p', 'r', 'o', 'g', 'r', 'a', 'm']
my_list[2: 5] = ['o', 'g', 'r']
my_list[2: -2] = ['o', 'g', 'r']
my_list[0: 3] = ['p', 'r', 'o']
```

افزودن عناصر به لیست پایتون

همانطور که قبلاً ذکر شد، لیست‌ها قابل تغییر هستند و می‌توانیم آیتم‌های یک لیست را تغییر دهیم

متد `append()`

برای اضافه کردن یک آیتم به انتهای لیست، می‌توانیم استفاده کنیم. برای مثال

```
In [2]: fruits = ['apple', 'banana', 'orange']
print('Original List:', fruits)

fruits.append('cherry')

print('Updated List:', fruits)
```

```
Original List: ['apple', 'banana', 'orange']
Updated List: ['apple', 'banana', 'orange', 'cherry']
```

افزودن عناصر در اندیس مشخص‌شده

متد `insert()`

می‌توانیم یک عنصر را در اندیس مشخص‌شده به یک لیست اضافه کنیم. برای مثال

```
In [3]: fruits = ['apple', 'banana', 'orange']
print("Original List:", fruits)

fruits.insert(2, 'cherry')
```

```
print("Updated List:", fruits)
```

Original List: ['apple', 'banana', 'orange']

Updated List: ['apple', 'banana', 'cherry', 'orange']

افزودن عناصر از سایر اشیای قابل تکرار به یک لیست

`extend()` متد

در لیست، تمام عناصر یک شیء قابل تکرار مشخص شده را به انتهای لیست اضافه می‌کند. برای مثال

```
In [4]: numbers = [1, 3, 5]
print('Numbers:', numbers)

even_numbers = [2, 4, 6]
print('Even numbers:', numbers)

# adding elements of one list to another
numbers.extend(even_numbers)

print('Updated Numbers:', numbers)
```

Numbers: [1, 3, 5]

Even numbers: [1, 3, 5]

Updated Numbers: [1, 3, 5, 2, 4, 6]

تغییر موارد فهرست

`=` عملگر

می‌توانیم با اختصاص مقادیر جدید، موارد یک لیست را تغییر دهیم. برای مثال

```
In [5]: colors = ['Red', 'Black', 'Green']
print('Original List:', colors)

# change the first item to 'Purple'
colors[0] = 'Purple'

# change the third item to 'Blue'
colors[2] = 'Blue'

print('Updated List:', colors)
```

Original List: ['Red', 'Black', 'Green']

Updated List: ['Purple', 'Black', 'Blue']

حذف یک مورد از فهرست

`remove()` متد

می‌توانیم آیتم مشخص شده را از یک لیست حذف کنیم. برای مثال

```
In [6]: numbers = [2,4,7,9]
```

```
# remove 4 from the list
numbers.remove(4)

print(numbers)
```

```
[2, 7, 9]
```

حذف یک یا چند عنصر از یک لیست

متد `del`

می‌توان برای حذف چندین عنصر یا حتی کل لیست استفاده کرد

```
In [7]: names = ['John', 'Eva', 'Laura', 'Nick', 'Jack']
```

```
# delete the item at index 1
del names[1]
print(names)

# delete items from index 1 to index 2
del names[1: 3]
print(names)

# delete the entire list
del names

# Error! List doesn't exist.
print(names)
```

```
['John', 'Laura', 'Nick', 'Jack']
```

```
['John', 'Jack']
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[7], line 15
     11 # delete the entire list
     12 del names
     13
     14 # Error! List doesn't exist.
--> 15 print(names)

NameError: name 'names' is not defined
```

طول لیست در پایتون

تابع `len()`

برای یافتن تعداد عناصر (طول) یک لیست، می‌توانیم استفاده کنیم. برای مثال

```
In [2]: cars = ['BMW', 'Mercedes', 'Tesla']

print('Total Elements:', len(cars))
```

Total Elements: 3

Method	Description
<code>append()</code>	Adds an item to the end of the list
<code>extend()</code>	Adds items of lists and other iterables to the end of the list
<code>insert()</code>	Inserts an item at the specified index
<code>remove()</code>	Removes the specified value from the list
<code>pop()</code>	Returns and removes item present at the given index
<code>clear()</code>	Removes all items from the list
<code>index()</code>	Returns the index of the first matched item
<code>count()</code>	Returns the count of the specified item in the list
<code>sort()</code>	Sorts the list in ascending/descending order
<code>reverse()</code>	Reverses the item of the list
<code>copy()</code>	Returns the shallow copy of the list

Grocery Shopping List Manager

```
In [1]: # Create an empty list to store grocery items
shopping_list = []

print("--- 🛒 Grocery Shopping List ---")

# 1. Adding items to the list (append)
shopping_list.append("Apples")
shopping_list.append("Milk")
shopping_list.append("Bread")
shopping_list.append("Eggs")

print("Added initial items!")
print("Current List:", shopping_list)
print("Total Items:", len(shopping_list))

# 2. Accessing items by position (indexing starts at 0)
first_item = shopping_list[0]
print(f"\nFirst item to pick up: {first_item}")

# 3. Removing an item once bought (remove)
```

```

bought_item = "Milk"
shopping_list.remove(bought_item)
print(f"\nBought {bought_item} and removed it from the list.")

# adding join method
print('- ' + '\n- ' .join(shopping_list))

# 4. Printing the final list using a simple loop
print("\n--- Remaining Items to Buy ---")
for item in shopping_list:
    print(f"- {item}")

print(f"\nItems left to buy: {len(shopping_list)}")

```

```

--- 🛒 Grocery Shopping List ---
Added initial items!
Current List: ['Apples', 'Milk', 'Bread', 'Eggs']
Total Items: 4

```

First item to pick up: Apples

Bought Milk and removed it from the list.

```

--- Remaining Items to Buy ---
- Apples
- Bread
- Eggs

```

Items left to buy: 3

دیکشنری پایتون

دیکشنری در پایتون مجموعه‌ای از اقلام است که به لیست‌ها و تاپل‌ها شباهت دارد. با این حال، برخلاف لیست‌ها و تاپل‌ها، هر قلم در یک دیکشنری یک جفت «کلید-مقدار» (شامل یک کلید و یک مقدار) است

ساخت دیکشنری

ما با قرار دادن جفت‌های «کلید: مقدار» درون آکولاد {} و جدا کردن آن‌ها با ویرگول، یک دیکشنری ایجاد می‌کنیم. برای مثال

```

In [9]: # creating a dictionary
country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
    "England": "London"
}

# printing the dictionary
print(country_capitals)

```

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'}
```

نکته:

- کلیدهای دیکشنری باید تغییرناپذیر باشند، مانند تاپل‌ها، رشته‌ها، اعداد صحیح و غیره. ما نمی‌توانیم از اشیای تغییرپذیر (مانند لیست‌ها) به عنوان کلید استفاده کنیم

`dict()`

- همچنین می‌توانیم با استفاده از تابع داخلی در پایتون، یک دیکشنری ایجاد کنیم

کلیدهای یک دیکشنری باید تغییرناپذیر باشند

اشیاء تغییرناپذیر پس از ایجاد، قابل تغییر (integer)، تاپل (tuple) و رشته (string) نیستند. برخی از اشیاء تغییرناپذیر در پایتون عبارت‌اند از اعداد صحیح

```
In [1]: # valid dictionary
# integer as a key
my_dict = {1: "one", 2: "two", 3: "three"}

# valid dictionary
# tuple as a key
my_dict = {(1, 2): "one two", 3: "three"}

# invalid dictionary
# Error: using a list as a key is not allowed
my_dict = {1: "Hello", [1, 2]: "Hello Hi"}

# valid dictionary
# string as a key, list as a value
my_dict = {"USA": ["Chicago", "California", "New York"]}
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[1], line 11
      7 my_dict = {(1, 2): "one two", 3: "three"}
      8
      9 # invalid dictionary
     10 # Error: using a list as a key is not allowed
--> 11 my_dict = {1: "Hello", [1, 2]: "Hello Hi"}
     12
     13 # valid dictionary
     14 # string as a key, list as a value

TypeError: unhashable type: 'list'
```

در این مثال، از اعداد صحیح، تاپل‌ها و رشته‌ها به عنوان کلید برای دیکشنری‌ها استفاده کرده‌ایم. هنگامی که از یک لیست به عنوان کلید استفاده کردیم، به دلیل ماهیت تغییرپذیر لیست، پیام خطایی رخ داد

کلیدهای یک دیکشنری باید یکتا باشند

کلیدهای یک دیکشنری باید منحصر به فرد باشند. اگر کلیدهای تکراری وجود داشته باشد، مقدار جدیدتر کلید، جایگزین مقدار قبلی می‌شود

```
In [2]: hogwarts_houses = {
    "Harry Potter": "Gryffindor",
    "Hermione Granger": "Gryffindor",
    "Ron Weasley": "Gryffindor",
    # duplicate key with a different house
    "Harry Potter": "Slytherin"
}

print(hogwarts_houses)
```

```
{'Harry Potter': 'Slytherin', 'Hermione Granger': 'Gryffindor', 'Ron Weasley': 'Gryffindor'}
```

در اینجا، کلید «هری پاتر» ابتدا به «گریفیندور» اختصاص می‌یابد. با این حال، ورودی دومی نیز وجود دارد که در آن هری پاتر به «اسلیترین» اختصاص داده شده است

از آنجا که وجود کلیدهای تکراری در دیکشنری مجاز نیست، آخرین ورودی (یعنی اسلیترین) جایگزین مقدار قبلی (گریفیندور) می‌شود

دسترسی به آیتم های دیکشنری

ما می‌توانیم با قرار دادن کلید درون کروشه، به مقدار یک عضو دیکشنری دسترسی پیدا کنیم

```
In [3]: country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
    "England": "London"
}

# access the value of keys
print(country_capitals["Germany"])    # Output: Berlin
print(country_capitals["England"])    # Output: London
```

```
Berlin
London
```

افزودن موارد به یک دیکشنری

ما می‌توانیم با اختصاص دادن یک مقدار به یک کلید جدید، عضوی به دیکشنری اضافه کنیم. برای مثال

```
In [4]: country_capitals = {
    "Germany": "Berlin",
    "Canada": "Ottawa",
}
```

```
# add an item with "Italy" as key and "Rome" as its value
country_capitals["Italy"] = "Rome"

print(country_capitals)
```

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'Italy': 'Rome'}
```

حذف موارد از دیکشنری

متد `del`

می‌توانیم برای حذف یک عنصر از دیکشنری استفاده کنیم. برای مثال

```
In [5]: country_capitals = {
        "Germany": "Berlin",
        "Canada": "Ottawa",
    }

# delete item having "Germany" key
del country_capitals["Germany"]

print(country_capitals)
```

```
{'Canada': 'Ottawa'}
```

متد `clear()`

می‌توانیم تمام عناصر یک دیکشنری را یکجا حذف کنیم

```
In [6]: country_capitals = {
        "Germany": "Berlin",
        "Canada": "Ottawa",
    }

# clear the dictionary
country_capitals.clear()

print(country_capitals)
```

```
{}
```

تغییر آیتم درون دیکشنری

دیکشنری‌های پایتون تغییرپذیر هستند. ما می‌توانیم با ارجاع به کلید یک عنصر در دیکشنری، مقدار آن را تغییر دهیم. برای مثال

```
In [7]: country_capitals = {
        "Germany": "Berlin",
        "Italy": "Naples",
        "England": "London"
    }
```

```
# change the value of "Italy" key to "Rome"
country_capitals["Italy"] = "Rome"

print(country_capitals)
```

```
{'Germany': 'Berlin', 'Italy': 'Rome', 'England': 'London'}
```

یافتن طول دیکشنری

تابع len()

می‌توانیم طول یک دیکشنری را پیدا کنیم

```
In [8]: country_capitals = {"England": "London", "Italy": "Rome"}
```

```
# get dictionary's length
print(len(country_capitals))    # Output: 2
```

```
numbers = {10: "ten", 20: "twenty", 30: "thirty"}
```

```
# get dictionary's length
print(len(numbers))            # Output: 3
```

```
countries = {}
```

```
# get dictionary's length
print(len(countries))          # Output: 0
```

2
3
0

متمدهای دیکشنری پایتون

Function	Description
<code>pop()</code>	Removes the item with the specified key.
<code>update()</code>	Adds or changes dictionary items.
<code>clear()</code>	Remove all the items from the dictionary.
<code>keys()</code>	Returns all the dictionary's keys.
<code>values()</code>	Returns all the dictionary's values.
<code>get()</code>	Returns the value of the specified key.
<code>popitem()</code>	Returns the last inserted key and value as a tuple.
<code>copy()</code>	Returns a copy of the dictionary.

نکته: تفاوت بین `pop()` و `remove()`

متد `pop`

- یک اندیس اختیاری را به عنوان ورودی می‌پذیرد تا مقدار موجود در آن جایگاه را حذف کند؛ در غیر این صورت، آخرین عنصر را حذف می‌کند

متد `remove`

- دقیقاً یک آرگومان می‌گیرد و نمی‌تواند خالی باشد

Student Grade Portal

```
In [4]: # 1. Initialize nested dictionary with student data
portal = {
    "S101": {
        "name": "Alice",
        "grades": {"Math": 95, "Science": 88, "History": 92}
    },
    "S102": {
        "name": "Bob",
        "grades": {"Math": 72, "Science": 79, "History": 68}
    }
}

# 2. Add a new student directly into the dictionary
portal["S103"] = {
    "name": "Charlie",
    "grades": {}
}
```

```

# 3. Add grades directly into Charlie's nested dictionary
portal["S103"]["grades"]["Math"] = 85
portal["S103"]["grades"]["Science"] = 90

# -----
# CALCULATIONS (Direct access using keys & values)
# -----

# Alice's calculations
alice_scores = portal["S101"]["grades"].values()
alice_avg = sum(alice_scores) / len(alice_scores)

# Bob's calculations
bob_scores = portal["S102"]["grades"].values()
bob_avg = sum(bob_scores) / len(bob_scores)

# Charlie's calculations
charlie_scores = portal["S103"]["grades"].values()
charlie_avg = sum(charlie_scores) / len(charlie_scores)

# Store all averages in a single dictionary
summary = {
    portal["S101"]["name"]: alice_avg,
    portal["S102"]["name"]: bob_avg,
    portal["S103"]["name"]: charlie_avg
}

# Find top performing student directly from the summary dictionary
top_student = max(summary, key=summary.get)

# -----
# DISPLAY OUTPUT (Direct printing)
# -----

print("=====")
print("🚪 STUDENT GRADE PORTAL")
print("=====")

# Student 1: Alice
print(f"ID: S101 | Name: {portal['S101']['name']}")
print(f"Grades : {portal['S101']['grades']}")
print(f"Average: {alice_avg:.2f}\n")

# Student 2: Bob
print(f"ID: S102 | Name: {portal['S102']['name']}")
print(f"Grades : {portal['S102']['grades']}")
print(f"Average: {bob_avg:.2f}\n")

# Student 3: Charlie
print(f"ID: S103 | Name: {portal['S103']['name']}")
print(f"Grades : {portal['S103']['grades']}")
print(f"Average: {charlie_avg:.2f}\n")

print("=====")
print("📊 SUMMARY DICTIONARY")
print("=====")

```

```
print(f"All Averages: {summary}")
print(f"Top Student : {top_student} ({summary[top_student]:.2f})")
print("=====")
```

```
=====
🎓 STUDENT GRADE PORTAL
=====
ID: S101 | Name: Alice
Grades : {'Math': 95, 'Science': 88, 'History': 92}
Average: 91.67

ID: S102 | Name: Bob
Grades : {'Math': 72, 'Science': 79, 'History': 68}
Average: 73.00

ID: S103 | Name: Charlie
Grades : {'Math': 85, 'Science': 90}
Average: 87.50

=====
📊 SUMMARY DICTIONARY
=====
All Averages: {'Alice': 91.66666666666667, 'Bob': 73.0, 'Charlie': 87.5}
Top Student : Alice (91.67)
=====
```

```
In [2]: # Same as above but using for loop
# Main dictionary containing student records and their nested subject grades
portal = {
    "S101": {
        "name": "Alice",
        "grades": {"Math": 95, "Science": 88, "History": 92}
    },
    "S102": {
        "name": "Bob",
        "grades": {"Math": 72, "Science": 79, "History": 68}
    },
    "S103": {
        "name": "Charlie",
        "grades": {"Math": 85, "Science": 90}
    }
}

# 1. Add a new student directly into the top-level dictionary
portal["S104"] = {
    "name": "Diana",
    "grades": {}
}
print(f"Added new student: {portal['S104']['name']} (ID: S104)")

# 2. Add grades directly to Diana's nested 'grades' dictionary
portal["S104"]["grades"]["Math"] = 91
portal["S104"]["grades"]["Science"] = 94
print(f"Added grades for Diana: {portal['S104']['grades']}\n")

# Dictionary mapping letter grades to cutoffs
```

```

grade_cutoffs = {90: "A", 80: "B", 70: "C", 60: "D", 0: "F"}

# Dictionary to store calculated averages for class summary
class_averages = {}

print("=====")
print("🎓 INDIVIDUAL STUDENT REPORT CARDS")
print("=====")

# 3. Iterate through the dictionary items (ID and student data)
for student_id, data in portal.items():
    print(f"\nStudent: {data['name']} (ID: {student_id})")
    print("-----")

    student_grades = data["grades"]

    if student_grades:
        # Calculate total score using dictionary values
        total_score = sum(student_grades.values())
        subject_count = len(student_grades)
        avg = total_score / subject_count

        # Save average to our summary dictionary
        class_averages[data["name"]] = avg

        # Print each subject and score
        for subject, score in student_grades.items():
            print(f"  - {subject:<10}: {score}")

        # Determine Letter grade using comparison Logic
        if avg >= 90:
            letter = grade_cutoffs[90]
        elif avg >= 80:
            letter = grade_cutoffs[80]
        elif avg >= 70:
            letter = grade_cutoffs[70]
        elif avg >= 60:
            letter = grade_cutoffs[60]
        else:
            letter = grade_cutoffs[0]

        print(f"  Average    : {avg:.2f}")
        print(f"  Letter      : {letter}")
    else:
        print("  No grades recorded.")

print("\n=====")
print("📊 CLASS-WIDE SUMMARY")
print("=====")

# Iterate through the averages dictionary we populated
for name, avg in class_averages.items():
    print(f"{name:<10} | Average: {avg:.2f}")

# Find top performing student using dictionary keys and get() method
top_student = max(class_averages, key=class_averages.get)

```

```
print("-----")
print(f"Top Student : {top_student} ({class_averages[top_student]:.2f})")
print("=====")
```

Added new student: Diana (ID: S104)

Added grades for Diana: {'Math': 91, 'Science': 94}

=====

🎓 INDIVIDUAL STUDENT REPORT CARDS

=====

Student: Alice (ID: S101)

- Math : 95
- Science : 88
- History : 92
Average : 91.67
Letter : A

Student: Bob (ID: S102)

- Math : 72
- Science : 79
- History : 68
Average : 73.00
Letter : C

Student: Charlie (ID: S103)

- Math : 85
- Science : 90
Average : 87.50
Letter : B

Student: Diana (ID: S104)

- Math : 91
- Science : 94
Average : 92.50
Letter : A

=====

📊 CLASS-WIDE SUMMARY

=====

Alice		Average: 91.67
Bob		Average: 73.00
Charlie		Average: 87.50
Diana		Average: 92.50

Top Student : Diana (92.50)

=====